

RV32I Processor

Three-stage pipeline

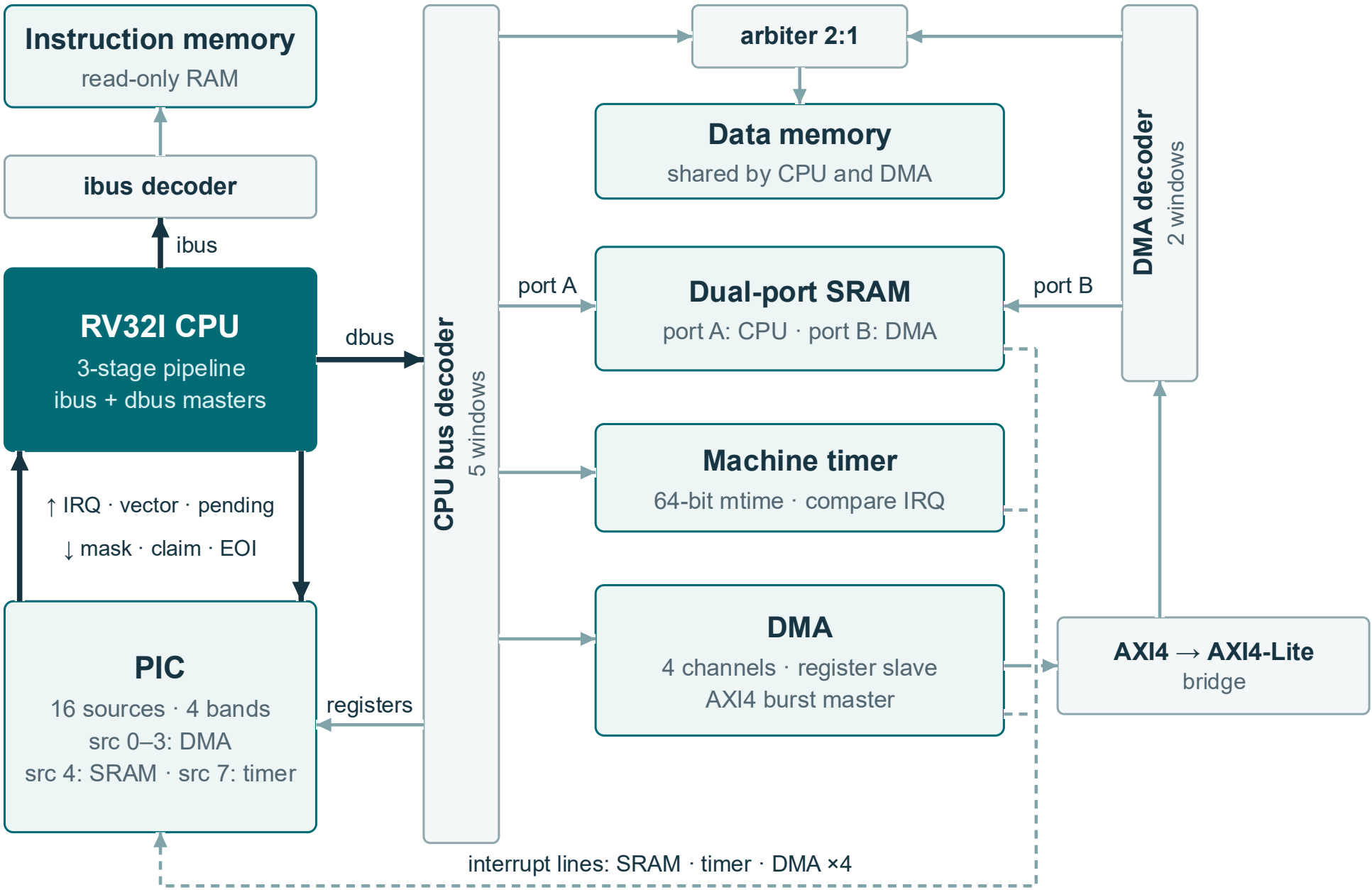
Bunea Cosmin-Andrei

Computer Engineering, year III

Academic year 2025–2026

Transilvania University of Braşov

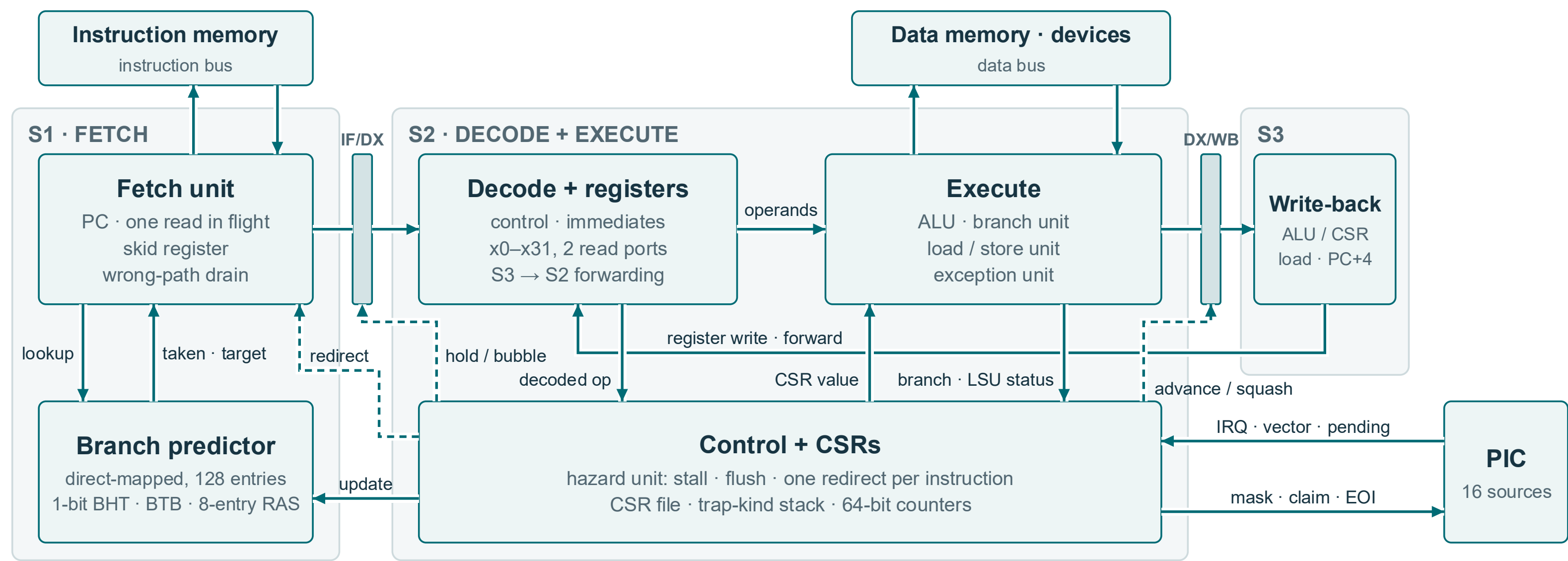
RTL implementation · interfaces · verification



The brief asks for a three-stage RV32I core with prediction and traps

- 01** Full RV32I instruction set; FENCE as a no-op, ECALL / EBREAK as traps
- 02** Three stages; a load or store completes inside S2
- 03** 32×32 -bit registers: 2 read ports, 1 write port, x0 = 0
- 04** Branch predictor with a 1-bit counter per entry
- 05** Exceptions and external interrupts, taken only between instructions
- 06** Machine-mode CSRs: mstatus, mie, mip, mtvec, mepc, mcause, mscratch
- 07** One central unit for every stall and flush
- 08** Two AXI4-Lite masters (instruction, data) and an interrupt link to the PIC

Three stages: S1 fetches, S2 decodes and executes, S3 writes back



THE BRIEF LEFT 23 QUESTIONS OPEN · THE FIVE THAT SHAPED THE PIPELINE

Predictor

Direct-mapped, 128 entries: index PC[8:2], tag PC[31:9]. 1-bit BHT + BTB + 8-entry RAS. Miss → not taken; trained at commit.

Stalls

Only a data access or WFI stalls S2. Load-use costs 0 cycles (S3 → S2 forwarding). S1 parks one word in a skid register.

Redirect order

Resolved in S2, one redirect per instruction: interrupt > exception > MRET > misprediction. Flush = valid bit cleared.

Nested traps

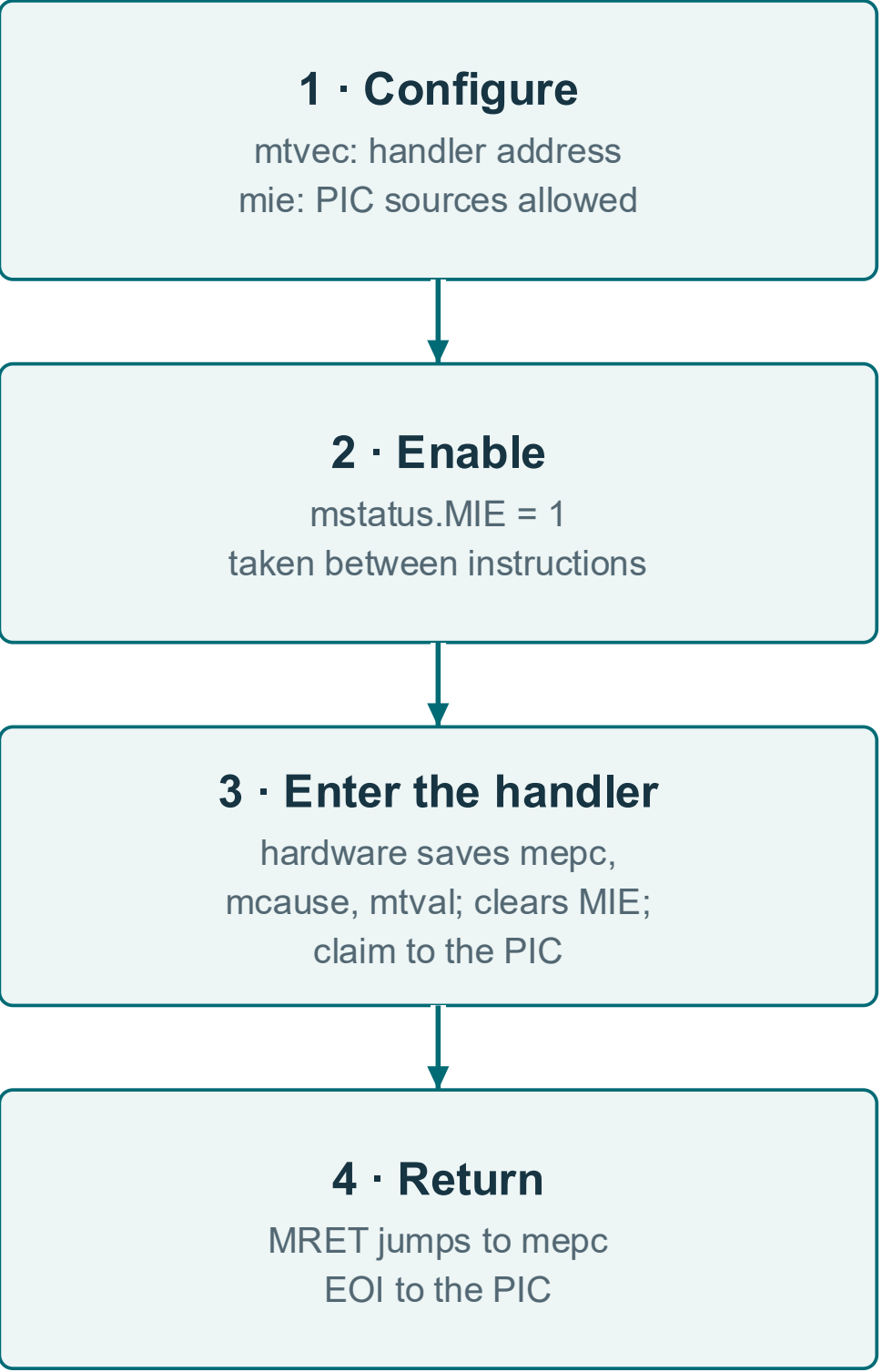
Entry saves mepc, mcause, mtval and MIE → MPIE. A 16-level trap-kind stack: MRET sends EOI only for an interrupt.

Precise errors

Fetch fault → cause 1, load / store fault → 5 / 7, misaligned → 4 / 6 before the bus, bad CSR → illegal. No write-back.

The CSRs connect software to traps, counters and the PIC

CSR	What it does
mstatus	MIE global enable; MPIE holds it during a trap, MRET restores it
mtvec	Handler base: direct, or vectored with one entry per interrupt
mepc · mcause · mtval	Return PC · cause, bit 31 = interrupt · faulting address or instruction
mscratch	Free word for the handler to save context
mie	One enable bit per PIC source; also drives the PIC mask
mip	Every source pending in the PIC; read-only
mcycle · minstret	64-bit cycle and retired-instruction counters; each half writable
mhpmcounter3–7	Event counters: mispredictions, fetch waits, data stalls, traps, WFI sleep
misa · mhartid · IDs	RV32I in misa; mhartid from HART_ID, tested with two harts



An independent model checks every instruction the core retires

THE CPU ALONE

11 programs vs the golden ISA model: 49,149 instructions, each checked at write-back

53 official riscv-tests pass (rv32ui + rv32mi), nominal and 40% backpressure

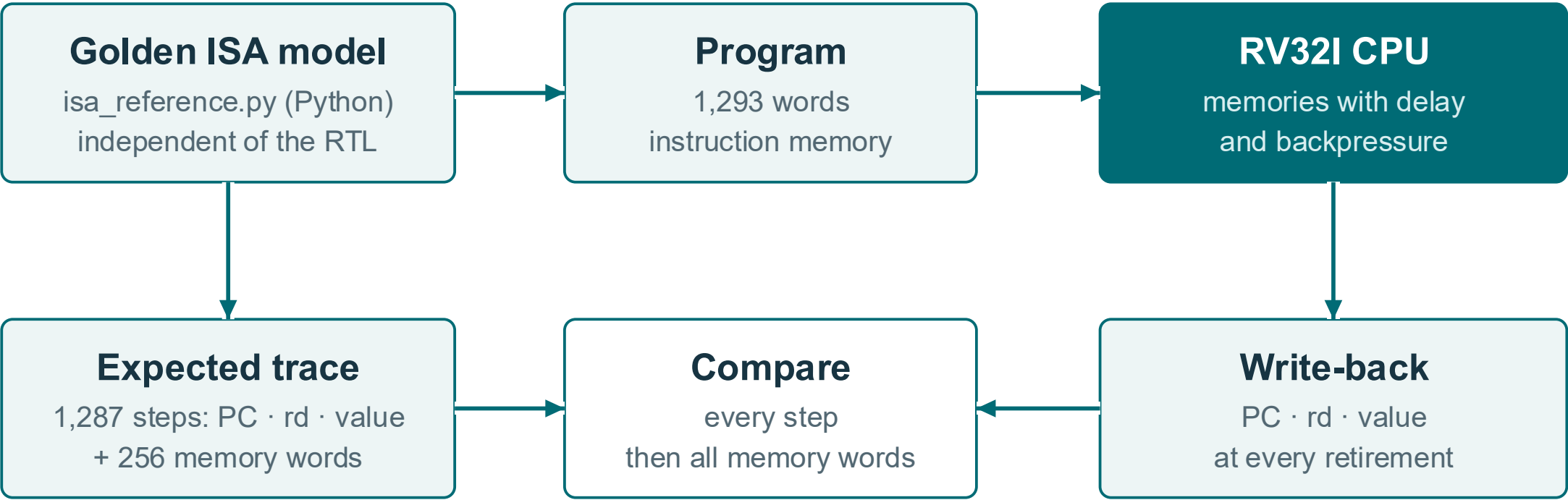
50 instruction kinds · all 32 shift amounts every byte lane · 256 memory words

925 directed checks: CSRs 293 · reset 217 counters 159 · traps 131 predictor 68 · ALU 57

37 SVA properties attached with bind: 17 pipeline / trap, 20 AXI protocol

90 coverage bins, all hit over 28 runs: instructions, traps, predictor, PIC claims

9 / 9 mutation testing: injected ALU, LSU, counter and trap defects, each caught



Nominal run: a byte store, then three loads of the same word

step	PC	instruction	model	CPU	
660	00000A50	sb x3, 1(x31)	no write	no write	✓
661	00000A54	lb x14, 1(x31)	x14 = FFFFFFFF	x14 = FFFFFFFF	✓
662	00000A58	lbu x15, 1(x31)	x15 = 000000FF	x15 = 000000FF	✓
663	00000A5C	lw x18, 0(x31)	x18 = 8000FFFF	x18 = 8000FFFF	✓

A nested interrupt opens two levels; each MRET closes one

THE CPU WITH THE PIC

382

checks, one program on CPU + PIC + mtimer: 9 exception causes, 8 interrupts, WFI wake-up; 4 memory timings

34

SoC runs, 767 checks; the CPU runs 16 programs: DMA, timer, PIC, isolation, reset, races

133

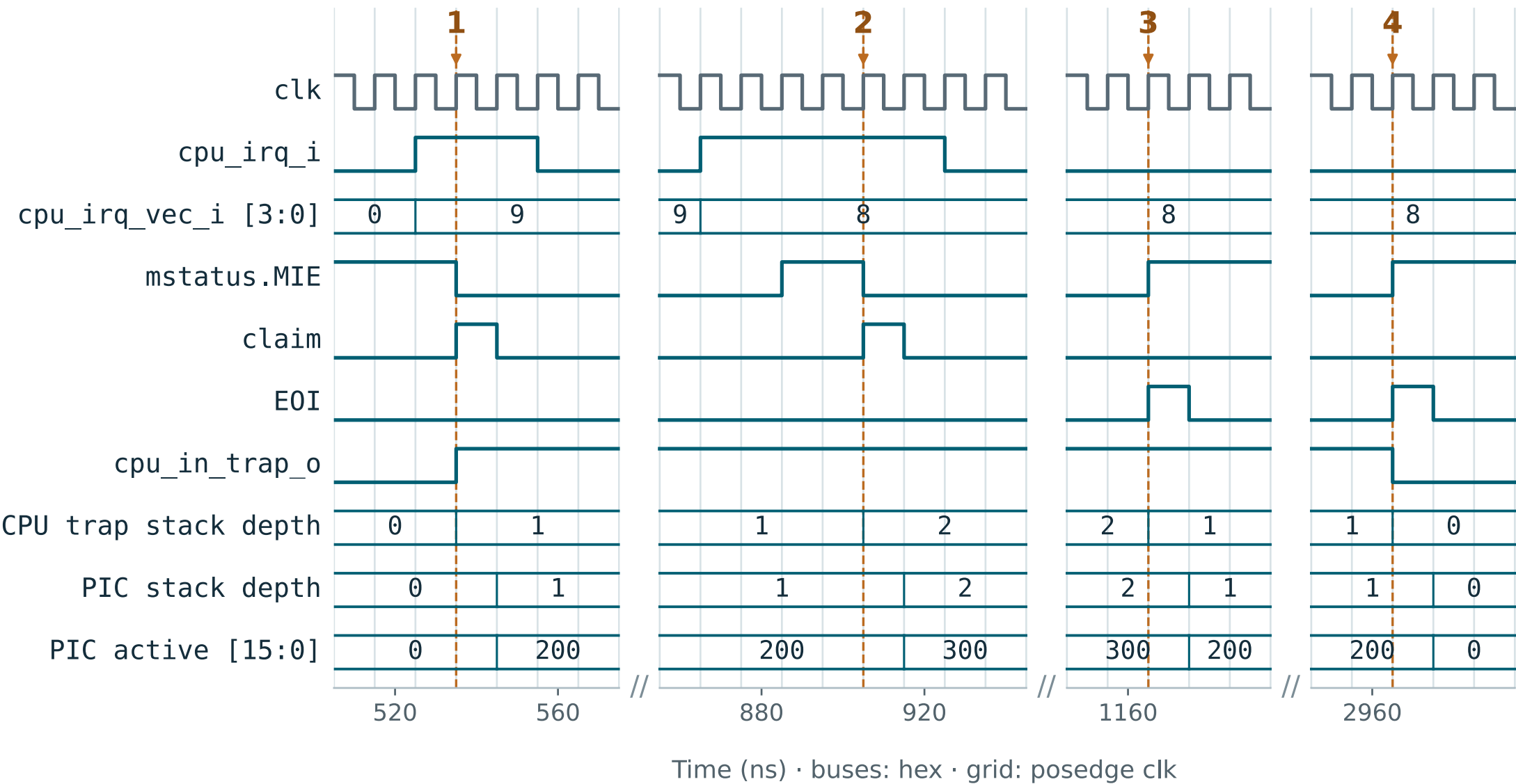
interrupt checks through the CPU: SW sources 36 · 16 levels 54 · nesting 16 · spurious 15 · escalation 12

29

CPU + PIC SVA properties bound in all 23 SoC benches, 51 Verilator runs

CI

every push: lint, all CPU / PIC / SoC benches with SVA, coverage and cover gates



1 · CPU takes source 9

claim pulses, cpu_in_trap rises, MIE drops to 0. Both stacks now hold 1 level; active = 200 (bit 9).

2 · Source 8 preempts

Handler 9 sets MIE back to 1. Source 8 is more urgent and gets claimed too: 2 levels, active = 300 (8 and 9).

3 · Handler 8 returns

Its MRET sends one EOI. Both stacks drop to 1 and active is back to 200; the CPU stays in the trap.

4 · Handler 9 returns

Its MRET sends the last EOI. Both stacks reach 0, active is 0 and cpu_in_trap falls.

Result: an integrated core, checked instruction by instruction

WHAT I BUILT

1. 3-stage RV32I: S3 → S2 forwarding, direct-mapped predictor, precise traps
2. CSR file, 64-bit counters, CPU–PIC link: mask, pending, claim, EOI
3. Golden ISA model with random programs, riscv-tests, SVA, mutation testing

WHAT I LEARNED

- Most of the thinking went into the moments when several things happen in one cycle: an interrupt, a stall and a wrong branch together. Writing their order down before the RTL made everything after it easier.
- Interrupts made me think about two blocks at once. The CPU and the PIC each keep their own stack, and nesting works because the two move in step, one level at a time.
- Writing the golden model in Python, apart from the Verilog, meant understanding every instruction twice: once for the model, once for the hardware.
- I stopped trusting a passing test by default. Injecting a defect into the RTL and watching the right test go red is what gave me confidence in the rest.

Thank you! Questions are welcome.